You must read this document carefully

Fall 2026
CMPS310 Software Engineering
- Group Project -

Milestone-2: Focused Design, Implementation, Testing and Scrum (10%)

This is a compulsory project. If you do not submit this, your grade will be zero (0) in this course.

Each group can have four members

Announcement Date: ***October 22, 2026***

Submission Due Date: 23:59 Saturday ***November 28, 2026***

1. Purpose and continuity with Milestone 1

Ms. Sara Ahmed has approved the continuation of the PearlMart Inventory Management System developed in Milestone 1. The original business requirements (Appendix A), together with the use-case specifications and class model developed in Milestone 1, remain the reference for the project. However, Milestone 2 focuses on the design and implementation of selected workflows rather than the complete system. Students are required to refine the relevant UML models, implement the specified workflows in a working Java desktop application, perform functional testing and non-functional testing, apply two appropriate software design patterns, and demonstrate the use of Scrum throughout development. Any justified changes to the Milestone 1 design must be documented and reflected consistently in the UML diagrams, implementation, and final report.

2. Required implementation scope

Required for Milestone 2	**Not Required for Milestone 2**
UC04 – Add Goods: Register a new inventory item or increase the quantity of an existing item. Validate the item information and quantity, and record each successful stock addition.	Implementation of other inventory operations, such as removing goods, managing supplier orders, identifying expired items, and generating administrative reports. (**Not required**)
UC13 – Process Sale: Process a sale containing one or more items, verify stock availability (UC14), calculate payable amounts including applicable discounts and taxes (UC15 and UC16), confirm the transaction, generate a SalesReceipt, and deduct inventory quantities exactly once.	Customer replacements, customer item requests, online payment integration, and other sales-related workflows outside UC13. (**Not required**)
UC17 – Cancel Current Receipt: Support cancellation of a pending sale as an alternative flow within UC13. Cancellation must leave inventory quantities unchanged and record the cancelled transaction.	A separate implementation or sequence diagram for UC17, or cancellation/refund processing after a sale has been completed. (**Not required**)
Shared Inventory: UC04 and UC13 must operate on the same inventory records within one running application. An in-memory data repository is acceptable, and persistent storage is optional.	Deployment across multiple computers, real-time synchronization between separate machines, production databases, and cloud deployment. (**Not required**)
Java Desktop Application: Develop one functional Java desktop application running on Windows 11, with a simple, usable GUI supporting both required workflows.	Web or mobile applications, Linux/macOS compatibility, and full login/password management. (**Not required**)

Validation and Transaction Consistency: Demonstrate input validation, correct inventory updates, recorded stock movements, completed sales, generated receipts, and preservation of inventory consistency when transactions fail or are cancelled.	Production-scale load testing, full backup/recovery infrastructure, and implementation of all 22 use cases. (**Not required**)

Students are required to use JavaFX, as introduced and practiced in the course laboratory sessions, to develop a simple, functional, and user-friendly desktop GUI. The interfaces must support the following operations:

1. **Add Goods (UC04):** Allow the user to register a new inventory item by entering its required information or to increase the quantity of an existing item. Display the updated inventory quantity and provide appropriate validation and error messages.
2. **Process Sale (UC13):** Allow the user to select one or more items, specify quantities, verify stock availability, and display item prices, subtotal, applicable discounts and taxes, and the final payable amount. The user must be able to confirm or cancel a pending sale. Upon successful confirmation, the system must generate and display a sales receipt and update the inventory quantities accordingly.

Both operations must use the same inventory records. The GUI should provide clear navigation between the required operations and meaningful feedback for successful and unsuccessful transactions. Advanced interface features, authentication screens, and interfaces for other use cases are not required.

3. References to agreed use cases and classes

Use the established Milestone 1 use-case names and identifiers. The two principal use cases to be implemented are 'UC04 – Add Goods' and 'UC13 – Process Sale'. 'UC13 – Process Sale' incorporates the behaviors of 'UC14 – Verify Sale Item Availability' and 'UC15 – Calculate Payable Amount', while 'UC15 – Calculate Payable Amount' invokes 'UC16 – Apply Discount and Tax Rules'. 'UC17 – Cancel Current Receipt' is handled as an alternative flow within 'UC13 – Process Sale' and does not require a separate implementation or sequence diagram. Appendix B provides the relevant use-case specifications and defines their implementation scope for this milestone.

Students must develop a focused design class diagram based on the relevant use-case specifications, their Milestone 1 analysis and design, and feedback received during the Milestone 1 oral examination. The class diagram should identify the classes necessary to implement the required functionality, including appropriate domain classes, classes responsible for coordinating system operations, and JavaFX user-interface classes. Additional helper or design-pattern classes may be introduced where justified. Students must specify the relevant attributes, operations, and relationships among these classes, ensuring consistency with their sequence diagrams and Java implementation. The selection of classes and assignment of responsibilities are design decisions that students must justify. A use case does not necessarily correspond to a single class.

4. Additional information provided by Ms. Sara

The following statements provide additional business and technical information about the PearlMart Inventory Management System. Some describe requirements for the eventual full system, while others apply directly to the limited prototype developed in Milestone 2. In Task 2, students must identify and classify the relevant system constraints and non-functional requirements (NFRs), define measurable acceptance criteria, and propose appropriate verification methods. Students must clearly distinguish between requirements that can be implemented and tested in the prototype and those that apply only to the eventual full system.

1. Platform and Hardware: The Milestone 2 prototype must be developed in Java, using JavaFX for the graphical user interface. For the eventual full system, PearlMart plans to use its existing computing infrastructure and can purchase no more than one additional server.
2. Inventory Size and Search Performance: PearlMart currently maintains approximately 20,000 product records, and this number is expected to increase to about 50,000. In the eventual full system, a normal item search should return results within two seconds under normal operating conditions,

even when the inventory reaches the expected size. Testing with the complete inventory size is not required for Milestone 2.

3. Concurrent Users: Approximately 20 employees may use the eventual system simultaneously, with the number expected to increase to 50 within three years. The full system should support this growth without unacceptable degradation in performance. Milestone 2 does not require implementation or testing of concurrent access by multiple users.
4. Inventory Consistency: The warehouse and billing counters must maintain a consistent view of inventory quantities. In the eventual full system, a successfully committed inventory change should become visible at other locations within two seconds. For Milestone 2, 'UC04 – Add Goods' and 'UC13 – Process Sale' must operate on the same inventory records within one running application, ensuring that stock changes made by either operation are immediately reflected in the other.
5. Transaction Integrity: The system must maintain accurate inventory quantities and transaction records. In the eventual full system, concurrent sales must not be allowed to deduct the same last available unit more than once. In the Milestone 2 prototype, successful stock additions and sales must update inventory quantities and corresponding records consistently. Failed operations must not leave partial updates, and cancelling a pending sale through 'UC17 – Cancel Current Receipt' must not permanently change inventory quantities. Implementing concurrent transaction processing is not required.
6. Security: Employee credentials and business transactions must be protected against unauthorized access. In the eventual full system, passwords must not be stored in readable form, and employees must only access functions permitted by their roles. Implementing authentication and role-based access control is not required for Milestone 2. Students should assume that the users performing the required operations are authorized and explain how the eventual system could address these security requirements.
7. Availability and Recovery: The eventual system is expected to remain available during PearlMart's operating hours. Regular backups must be performed so that recovery from a failure results in the loss of no more than one business day of information. Implementing backup, recovery, or high-availability infrastructure is not required for Milestone 2. Students should identify the limitations of their chosen storage approach.
8. Modifiability: PearlMart may introduce new item categories, discount and tax rules, and reporting functions in future releases. The system should be designed so that changes to one business rule or component require minimal modifications to unrelated parts of the software. The Milestone 2 design should demonstrate appropriate modularity, separation of responsibilities, and limited dependencies between components.
9. Interoperability: In future releases, PearlMart may exchange purchase orders and invoices with external suppliers using different data formats or communication methods. Changes to supplier communication mechanisms should have minimal impact on inventory and sales processing. Supplier integration is outside the implementation scope of Milestone 2.
10. Usability: The JavaFX interface developed for Milestone 2 must support the required operations of 'UC04 – Add Goods' and 'UC13 – Process Sale' through clear and straightforward screens. Users should be able to enter the required information, review the relevant item and transaction details, and receive meaningful confirmation or error messages. The interface must support confirming and cancelling a pending sale without unnecessary navigation. Advanced graphical features are not required.
11. Prototype Performance Testing: For a feasible local performance evaluation, a prototype containing at least 1,000 preloaded inventory item records should complete each of ten successive item lookups within two seconds on the specified Windows 11 test computer. The response time is measured from submitting the lookup request until the corresponding result is displayed. Teams selecting performance as their experimentally verified NFR may use this criterion. This prototype-level evaluation does not replace the full-system performance requirement described in Statement 2.

5. Assessed tasks and allocation

Task	Assessment focus	Weight
1	Object-Oriented Design: Two UML sequence diagrams for 'UC04 – Add Goods' and 'UC13 – Process Sale', a focused design class diagram developed by the students, and a small architectural sketch.	15%
2	System Constraints and NFRs: Identify and classify relevant system constraints and specify three measurable non-functional requirements, distinguishing prototype requirements from full-system requirements.	10%
3	Java Implementation: Implement 'UC04 – Add Goods' and 'UC13 – Process Sale', including their required supporting behaviors, using Java and JavaFX with shared inventory data.	20%
4	Software Testing: Choose one of two alternatives: (A) functional testing of both UC04 – Add Goods and UC13 – Process Sale, or (B) functional testing of one of these use cases and testing of one measurable NFR.	20%
5	Software Design Patterns: Select, implement, and demonstrate TWO appropriate design patterns from two different categories (creational, structural, or behavioral). Singleton is not counted as one of the two design patterns.	15%
6	Scrum and GitHub: Demonstrate Scrum practices through two Sprints, using GitHub Issues and Projects for planning, tracking, code collaboration, reviews, and evidence of individual contributions.	20%
	TOTAL	100%

Task 1 – Object Interaction and Focused Class Design (15%)

- Develop two UML sequence diagrams using Visual Paradigm: one for **UC04 – Add Goods** and another for **UC13 – Process Sale**. The diagrams must reflect the normal scenarios and relevant alternative flows described in Appendix B, including the supporting use cases where applicable.
- Develop a focused UML design class diagram identifying the classes required to implement the selected use cases. Include appropriate attributes, operations, relationships, and multiplicities. Students are expected to identify the classes and assign their responsibilities independently, based on their Milestone-1 work and the provided requirements.
- Ensure consistency among the sequence diagrams, class diagram, and implementation. Explain any significant changes from Milestone 1.

Grading (15 marks): Sequence diagram for UC04 (5); sequence diagram for UC13 (5); focused class diagram (5).

Task 2 – System Constraints and Non-Functional Requirements (10%)

- Identify and classify at least four relevant system constraints using the information provided in Section 4. Explain their implications for the proposed system.
- Specify exactly three non-functional requirements (NFRs). For each, identify its quality attribute, source requirement, measurable acceptance criterion, and proposed verification method.
- Distinguish between requirements applicable to the complete system and those that can be verified using the Milestone 2 prototype.
- Document and justify any additional assumptions.

Grading (10 marks): Identification and classification of constraints (3); specification of three measurable NFRs (6); assumptions and scope distinction (1).

Task 3 – Java Implementation of the Selected Use Cases (20%)

- Implement **UC04 – Add Goods** and **UC13 – Process Sale**, including their supporting behaviors and relevant alternative flows, according to the use-case specifications provided in Appendix B.
- Develop a functional Java desktop application using JavaFX. The GUI must support both use cases, allow users to perform the required operations, display their results, and provide appropriate feedback.
- Both use-cases must operate on the same inventory records. The implementation must maintain inventory and transaction consistency, including when operations fail or are cancelled.
- Demonstrate the successful execution of both use cases, including their interaction with the shared inventory.

Grading (20 marks): Implementation of UC04 (7); implementation of UC13 and supporting behaviors (9); validation and transaction consistency (2); JavaFX interface and demonstration (2).

Task 4 – Software Testing (20%)

Each team must select ONE of the following testing routes. Both routes carry the same assessment weight.

Route A – Functional Testing of Two Use Cases

- Test both **UC04 – Add Goods** and **UC13 – Process Sale**.
- Provide at least six functional test cases, with at least three for each use case, covering normal and alternative scenarios.

Route B – Functional Testing of One Use-Case and One NFR

- Select either **UC04 – Add Goods** or **UC13 – Process Sale** and provide at least four functional test cases covering normal and alternative scenarios.
- Select one measurable NFR specified in Task 2 and experimentally verify it using the implemented prototype.

General Testing Requirements

- Apply appropriate software testing techniques covered in the course.
- Document test cases, test data, expected results, actual results, and pass/fail decisions.
- Provide sufficient evidence of test execution and discuss the results.
- For NFR testing, document the test environment, measurement procedure, results, and conclusions.

Grading (20 marks for either route): Test selection, coverage, and techniques (7); test-case documentation and results (9); evidence and evaluation (4).

Task 5 – Application of Two Software Design Patterns (15%)

- Select and implement two appropriate software design patterns from two different categories: creational, structural, or behavioral. The Singleton pattern is not permitted.
- For each pattern, identify its category, justify its selection, and explain the design problem it addresses within the implemented prototype.
- Provide an appropriate UML class diagram illustrating the pattern and identify its implementation in the Java code.
- Demonstrate the actual use of both patterns in the working application.

Grading (15 marks): First pattern - justification (2), UML design (2), implementation (3); second pattern - justification (2), UML design (2), implementation (3); consistency and integration (1).

Task 6 – Application of Scrum Using GitHub Issues and Projects (20%)

- Continue using the GitHub repository and GitHub Project established in Milestone 1.
- Organize Milestone 2 development into two Sprints, each no longer than two weeks and completed before the submission deadline.
- Maintain a prioritized Product Backlog and Sprint Backlogs using GitHub Issues and Projects. Document Sprint Goals, task assignments, estimates, acceptance criteria, and progress.
- Apply appropriate GitHub collaboration practices, including commits, branches, pull requests, and peer reviews.
- Document Sprint Reviews and Retrospectives, highlighting progress, challenges, and improvements.
- Provide evidence of Scrum activities throughout development and demonstrate meaningful contributions from all four team members.

Grading (20 marks): Sprint planning (4); backlog and issues (4); progress tracking and reviewed pull requests (7); Sprint Reviews and Retrospectives (3); individual contribution evidence (2).

6. Required Submission

Each team must submit the following deliverables:

1. **Project Report:** A concise Word document containing the two UML sequence diagrams, focused design class diagram, system constraints and NFRs, testing procedures and results, justification and UML diagrams for the two selected design patterns, and a summary of Scrum activities.
2. **UML Models:** The original Visual Paradigm (.vpp) file(s) containing the UML diagrams presented in the report. The models must be consistent with the implemented system.
3. **Java Implementation:** The complete Java and JavaFX project, including source code, required dependencies, sample data, and clear instructions for compiling and running the application.
4. **Testing Evidence:** Documented test cases and results corresponding to the selected testing route (A or B), supported by appropriate evidence of execution.
5. **Scrum and GitHub Evidence:** Links to the assigned GitHub repository, GitHub Project, Milestone 2 GitHub milestone, and final submission commit. Include evidence of Sprint planning, progress tracking, reviews, retrospectives, and individual contributions throughout both Sprints.
6. **Individual Peer Evaluation:** Each team member must independently submit a peer evaluation indicating the estimated percentage contribution of every team member, including themselves. The percentages must total 100%. These evaluations may be considered when determining individual project marks. Each evaluation must be submitted separately through Blackboard and must not be included in the shared project report or GitHub repository.
7. All final deliverables must be committed to a **Milestone-2** folder on the main branch of the assigned GitHub repository and submitted according to the instructions provided by the course team through Blackboard.

Important: Students must ensure consistency among the use-case specifications, UML models, Java implementation, design patterns, and testing results. The submitted application must be executable, and the Visual Paradigm files must open correctly. Scrum evidence must reflect activities performed throughout the development process, rather than being prepared only at the end of the project.

Appendix A – Original Milestone 1 business/system description

The original business requirements continue to define the PearlMart system. The assessment tasks and submission rules for Milestone 2 are those in the main body of this document. The following text is reproduced from the earlier draft's Appendix A.

Background

Assume that Ms. Sara Ahmed has recently been appointed as the operations manager of PearlMart, a medium-sized retail store. She has contacted your team to design a software system exclusively for managing the store's inventory activities.

PearlMart currently relies heavily on manual procedures to maintain information about goods. This has made it difficult to search for products, determine available quantities, update prices, identify expired goods, and retrieve previous transaction records. Manual processing has also resulted in occasional human errors, inconsistent records, and loss of information. Ms. Sara therefore requests the development of an automated PearlMart Inventory Management System that operates according to the requirements described below.

The system will support activities performed both in the warehouse and at the store's billing counters. Although the system can be accessed from different locations within PearlMart, it is considered one integrated system, and all inventory information should remain consistent.

Each item maintained by PearlMart has a unique item identifier and barcode. Information recorded about an item includes its name, selling price, available quantity, manufacturer, place of origin, and, when applicable, its expiration date. Additional comments about an item may also be recorded.

Several types of employees use the system. An Administrator performs administrative activities and manages important changes to system information. An Inventory Manager monitors the inventory and manages inventory-related transactions. A Store Manager supervises the receipt and storage of goods in the warehouse. Salespersons use the system mainly at the billing counters to process customer purchases and related transactions.

All authorized users must log in before using the system. A user enters a username and password, and the system verifies the credentials. If the credentials are valid, the user is given access to the functions appropriate to his/her responsibilities. Authorized users may also change their passwords.

When new goods arrive at PearlMart, an authorized employee scans the item's barcode or enters its identifier. If the item is new, the employee enters the required information about the item, including its name, selling price, quantity, manufacturer, place of origin, and expiration date when applicable. The system then creates a new inventory record. If the item already exists, its available quantity is increased accordingly.

Goods may also need to be removed from the inventory. An authorized employee identifies the item and specifies the quantity to be removed. The system retrieves the corresponding item information and checks whether the requested quantity is available. If the request can be processed, the system reduces the available quantity and records the change.

For goods that have expiration dates, the system keeps track of expiration information. Authorized employees can request information about expired goods. The system identifies the expired items and displays their details. The employee may then remove the appropriate goods from the inventory.

Authorized employees can search for an item using its identifier, barcode, or name. The system retrieves and displays the corresponding information, including the current price and available quantity. Certain item information may also be updated when necessary. For example, when the selling price of an item changes, an authorized employee retrieves the item, enters the new price, and the system updates the item record.

PearlMart obtains goods from external Suppliers. When additional goods are required, the Inventory Manager prepares an order specifying the required item, quantity, delivery address, contact information, and any relevant comments. The system records the order and forwards the necessary order information to the selected Supplier.

The Supplier processes the request and provides information concerning the ordered goods and the corresponding invoice. The invoice contains information such as the ordered items, quantities, prices, and total amount. When the ordered goods are delivered to PearlMart, the Store Manager verifies the received goods. The system records the received quantities and updates the inventory accordingly.

At the billing counter, a Salesperson processes the items selected by a customer. For each item, the Salesperson scans the item's barcode or enters its identifier and specifies the quantity being purchased. The system retrieves the item information and checks whether the requested quantity is available.

For every valid item, the system determines its selling amount. Where applicable, discounts and taxes are calculated. After all selected items have been processed, the system calculates the final amount to be paid by the customer.

The system then generates a receipt containing relevant transaction information, including the purchased items, quantities, prices, discounts, taxes, and final amount. When the transaction is successfully completed, the quantities of the purchased items are deducted from the inventory.

Before a billing transaction is completed, the Salesperson may cancel the current receipt. If the receipt is cancelled, the current transaction is terminated and the quantities of the items involved must not be permanently deducted from the inventory. The system records the appropriate information concerning the cancelled transaction.

PearlMart also allows customers to request replacement of purchased items according to store policy. To process a replacement, the Salesperson enters the required transaction and item information. The system retrieves the corresponding information and verifies the request. If the replacement is accepted, the system records the replacement and makes the necessary updates to the inventory.

A customer may sometimes request an item that is not currently available at PearlMart. In this case, an authorized employee records the requested item and the required quantity. The system maintains these customer requests so that the Inventory Manager can review them when deciding which goods should be ordered from Suppliers.

The system keeps track of important inventory activities and transactions, including goods added to or removed from inventory, goods received from Suppliers, supplier orders, sales transactions, cancelled receipts, replacement transactions, customer requests, and expired goods.

Authorized managers can also request summary reports about PearlMart's inventory activities. Depending on the type of report requested, the system retrieves the relevant information and generates a summary of the corresponding transactions and inventory records.

Ms. Sara expects PearlMart's inventory activities to be automated to the greatest extent possible. Information maintained by the system should remain consistent whether an operation is performed from the warehouse or from a billing counter.

The Supplier is external to the PearlMart Inventory Management System. Your analysis should therefore include the interaction between PearlMart and its Suppliers, but you should not analyze or design the internal operations of a Supplier's own system.

Additional Information

You will likely need additional information about the system's operation to complete this project. In software development, requirements may initially be incomplete or unclear, and software engineers often need to seek clarification or make reasonable assumptions.

For this project, your Lab Instructor is your client, and you may direct questions concerning the system requirements to him/her. You may also make logical assumptions whenever necessary to complete your analysis and design.

List and justify assumptions, identify affected models, and apply published course clarifications consistently.

Appendix B – Implementation details and instructor-supplied use case specifications

The following minimum behaviors are assessed in this milestone.

B.1 UC04 – Add Goods (InventoryManager)

Behavior	Required observable outcome
New item	When an identifier/barcode is new, capture required item information and a valid positive quantity; create InventoryItem and one corresponding InventoryMovement.
Existing item	When the item exists, increase availableQuantity by the accepted positive quantity and create one corresponding InventoryMovement.
Invalid entry	Reject invalid or duplicate item identification, missing mandatory fields and nonpositive quantities without a partial stock change.

B.2 UC13 – Process Sale (Salesperson)

Behavior	Required observable outcome
Build purchase	Identify one or more items and requested positive quantities; check the current shared quantities (UC14) and retain transaction-time unit prices.
Calculate payable amount	Calculate line/subtotals and applicable discount/tax adjustments using UC15 and UC16; use documented policy assumptions (including zero when not applicable).
Confirm sale	Only after successful confirmation, record a completed sale, generate SalesReceipt, deduct stock once and record stock movements.
Reject or cancel	Unknown items, invalid/insufficient quantity and cancellation while pending (UC17) must leave no permanent stock deduction.

B.3 Relevant use-case specifications

Note the use-case ID was based on the model answer numbering of use-cases. The ID is not that important, the name of the use-case is important.

UC04 – Add Goods

Use case ID or name	UC04 – Add Goods
Brief Description	Create a new inventory item record or increase the quantity of an existing item.
Primary actors	Inventory Manager (assumed authorized role).
Trigger(s)	An authorized stock addition is requested when goods are available to be registered.
Preconditions	• The Inventory Manager is authenticated and authorized. • Item identification and a positive quantity are available.
Post-conditions	• Success: a new item exists or the existing item quantity increases; an inventory movement is recorded. • Failure: existing inventory quantities remain unchanged.

Normal Scenario

Actor Action	System Response
1. Selects Add Goods and scans a barcode or enters an item identifier.	1. Searches for an item with the supplied identifier/barcode.
2. If the item is new, enters name, selling price, manufacturer, origin, quantity, optional expiration date, and comments.	2. Validates required fields, identifier/barcode uniqueness, and quantity.
3. If the item exists, enters the quantity being added.	3. Retrieves existing item details and calculates the proposed new quantity.
4. Confirms the addition.	4. Creates the item or updates its available quantity and records a corresponding inventory movement.
5. Reviews the result.	5. Displays the updated item information and confirmation.

Alternative flows:

2a. New item has missing/invalid information or duplicate identification: reject until corrected.

3a. Quantity is zero or negative: reject the addition.

4a. Update cannot be committed consistently: keep stock unchanged and display failure.

Non Functional Requirements (Optional):

Data Consistency: Inventory quantities and corresponding movement records must be updated consistently.

Data Integrity – Duplicate Prevention: Goods already received through UC11 (Receive Supplier Goods) must not be added to inventory a second time

UC13 – Process Sale

Use case ID or name	UC13 – Process Sale
Brief Description	Process customer-selected items, calculate the final amount, generate a completed receipt, and update stock upon successful sale completion.
Primary actors	Salesperson.
Trigger(s)	A customer presents one or more items for purchase at a billing counter.
Preconditions	• Salesperson is authenticated and authorized. • The integrated inventory service is available.
Post-conditions	• Completed sale: final receipt and transaction are saved, and purchased quantities are deducted exactly once. • Cancelled/unsuccessful sale: no permanent stock deduction; cancellations are recorded by UC17.

Normal Scenario

Actor Action	System Response
1. Starts a new billing transaction.	1. Creates a pending sale record/context.
2. Scans a barcode or enters an item identifier and quantity.	2. Invokes UC14 to retrieve item/price and verify requested stock.
3. Adds the remaining selected items.	3. Builds the validated sale list and identifies any invalid lines.
4. Requests the amount payable.	4. Invokes UC15 (which evaluates UC16) and displays item amounts, discounts, taxes, and total.
5. Reviews total and confirms that the transaction has been successfully completed.	5. Rechecks available stock as necessary, finalizes the transaction, and deducts purchased quantities.
6. Requests the receipt.	6. Generates/stores the final receipt containing purchased items, quantities, prices, discounts, taxes, and final amount.
7. Finishes the interaction.	7. Displays the receipt and completion confirmation.

Alternative flows:

2a. Unrecognized item: report it and allow correction/removal of the invalid line.

2b. Insufficient stock: reject that quantity; do not finalize until resolved.

4a. Invalid calculation inputs/rules: do not finalize until the payable amount is valid.

5a. Salesperson requests cancellation before completion: invoke UC17 as an extension; do not deduct stock.

5b. Stock changes before finalization or transaction commit fails: reject/roll back finalization and retain accurate stock.

Non Functional Requirements (Optional):

Data Consistency – Cross-Location Consistency: Ensure that sales transactions and inventory updates remain consistent across the warehouse and billing counters.

UC14 – Verify Sale Item Availability

Use case ID or name	UC14 – Verify Sale Item Availability
Brief Description	Retrieve item data and check that a requested purchase quantity is available for UC13.
Primary actors	Salesperson (indirect, via UC13).
Trigger(s)	Process Sale needs item and stock validation for a scanned/entered line.
Preconditions	• A pending sale exists. • An item identifier/barcode and requested quantity have been entered.
Post-conditions	• Success: a validated sale line (item, price, quantity) is returned to UC13. • Failure: the invalid line is rejected without changing stock.

Normal Scenario

Actor Action	System Response
1. Provides item identifier/barcode and purchase quantity within UC13.	1. Locates the inventory item.
2. Requests line validation.	2. Verifies quantity is positive and does not exceed available stock.
3. Continues the sale.	3. Returns item details, selling price and validated quantity to UC13.

Alternative flows:

1a. Item not found: return an invalid-item result.
2a. Quantity is nonpositive or exceeds stock: return validation error and currently available quantity.

Non Functional Requirements (Optional):

Data Consistency – Real-Time Inventory Accuracy: Ensure that item availability checks use the latest committed inventory quantities shared across all locations, rather than outdated local copies.

UC15 – Calculate Payable Amount

Use case ID or name	UC15 – Calculate Payable Amount
Brief Description	Calculate individual selling amounts and the final payable amount for the validated sale lines.
Primary actors	Salesperson (indirect, via UC13).
Trigger(s)	Process Sale requests the total for its selected items.
Preconditions	• At least one valid sale line exists. • Item quantities and prices are available.
Post-conditions	• Item amounts, applicable discounts/taxes, and final amount are returned to UC13; stock remains unchanged.

Normal Scenario

Actor Action	System Response
1. Requests a price calculation during UC13.	1. Calculates each line amount using selected quantity and selling price.
2. Reviews transaction calculation.	2. Invokes UC16 to evaluate applicable discount and tax rules.
3. Requests the final payable amount.	3. Combines line amounts and applicable adjustments and returns the final total to UC13.

Alternative flows:

1a. No valid sale lines: return an error; do not generate a payable total.
2a. Policy data cannot be evaluated: signal a calculation failure rather than invent amounts.

Non Functional Requirements (Optional):

Data Consistency – Transaction-Time Pricing: Ensure that all payable amount calculations use a consistent set of item prices applicable at the time of the transaction.

UC16 – Apply Discount and Tax Rules

Use case ID or name	UC16 – Apply Discount and Tax Rules
Brief Description	Determine and calculate discounts and taxes applicable to a pending sale.
Primary actors	Salesperson (indirect, through UC13 → UC15).
Trigger(s)	UC15 requests discount and tax evaluation.
Preconditions	• Valid sale amounts are available. • Applicable store rules are defined for the model (assumption A9).
Post-conditions	• Discount and tax amounts (including zero when not applicable) are returned to UC15; inventory is unchanged.

Normal Scenario

Actor Action	System Response
1. Initiates sale amount calculation through UC15.	1. Retrieves the applicable store discount and tax rules.
2. Provides the pending sale details through UC15.	2. Determines which rules apply and calculates the corresponding adjustments.
3. Continues total calculation.	3. Returns discount and tax amounts to UC15.

Alternative flows:

1a. No discount or tax applies: return zero for the relevant amount.
2a. Required policy is missing or invalid: return a validation error for resolution.

Non Functional Requirements (Optional):

Maintainability – Configurability: Discount and tax rates and calculation rules should be configurable rather than hard-coded (design assumption).

UC17 – Cancel Current Receipt

Use case ID or name	UC17 – Cancel Current Receipt
Brief Description	Cancel a pending sales transaction before successful completion, recording cancellation without a permanent stock deduction.
Primary actors	Salesperson.
Trigger(s)	During UC13, the Salesperson requests cancellation of the current pending receipt.
Preconditions	• Salesperson is authenticated and authorized. • The transaction has not yet been successfully completed.
Post-conditions	• A cancelled transaction is recorded and its pending work is terminated. • Inventory quantities are unchanged, and no completed sales receipt is issued.

Normal Scenario

Actor Action	System Response
1. Selects Cancel Current Receipt while UC13 is pending.	1. Checks that the transaction is not completed.
2. Confirms cancellation.	2. Marks the pending sale as cancelled and records its transaction details.
3. Requests the result.	3. Discards any uncommitted purchase effects and displays cancellation confirmation.

Alternative flows:

1a. Sale is already completed: this pending-sale cancellation use case is unavailable; any later return follows separate policy.

2a. Salesperson declines cancellation: resume the pending sale without stock changes.

2b. Cancellation recording fails: report the failure and do not silently mark a transaction completed.

Non Functional Requirements (Optional):

Data Integrity – Transaction Atomicity: Ensure that cancelling a sales transaction leaves inventory quantities unchanged, with no permanent stock deductions.

Appendix C – Individual Peer Evaluation Form

Each student must independently evaluate the contributions of all four team members, including themselves. Assign a percentage representing each member's estimated contribution to the project. **The total must equal 100%.**

These evaluations may be used to adjust the individual marks awarded for the project.

Evaluator Information

Student ID: _____

Student Name: _____

Group Number: _____

Contribution Evaluation

Student ID	Student Name	Contribution (%)

Comments (Optional):

Important: Submit this form individually through Blackboard. Do not share your evaluation with other team members or include it in the group's GitHub repository.